MOCK EXAM · พร้อมเฉลยละเอียดท้ายเล่ม

ข้อสอบเสมือนจริง
Digital Logic + Computer Architecture

ชุดที่ 1 ตามเกณฑ์จริง (Choice 40 + ตอบสั้น 10 = 60 คะแนน) · ชุดที่ 2 ฝึกเพิ่มแบบข้อเขียนยาว

ชุดที่ 1	90 นาที · 60 คะแนน · Digital Logic 15 / Com Arch 45 · หน้าโจทย์ถัดไป
ชุดที่ 2	90 นาที · 100 คะแนน · Com Arch อย่างเดียว เน้นการคำนวณหลายขั้นตอน
เฉลย	อยู่ท้ายเล่ม: เฉลยชุดที่ 1 ตามด้วยเฉลยชุดที่ 2 (มีเหตุผลทุกข้อ และบอกว่าตัวเลือกไหนเป็นตัวลวง)

วิธีใช้: จับเวลาจริง ห้ามดูโน้ต ใช้เครื่องคิดเลขได้ ผิดไม่ติดลบ จึงควรตอบทุกข้อ ทำเสร็จแล้วค่อยเปิดเฉลย
ข้อตกลงทั้งเล่ม: byte addressing · 1 word = 4 bytes · 1 KB = 2^{10} B
ข้อสอบชุดนี้แต่งขึ้นตามแนวของสไลด์และ Exercise ท้ายบท ไม่ใช่ข้อสอบเก่า

ชุดที่ 1: ข้อสอบเสมือนจริงตามเกณฑ์ (60 คะแนน, 90 นาที)

ส่วน	จำนวน	คะแนน	Digital Logic	Computer Architecture
Part 1: Choice	40 ข้อ × 1	40	ข้อ 1–11	ข้อ 12–40
Part 2: ตอบสั้น	10 ข้อ × 2	20	S1–S2	S3–S10
รวม		**60**	**15**	**45**

Part 1: Choice (ข้อละ 1 คะแนน)

Digital Logic (ข้อ 1–11)

1. $(1011.01)_2$ มีค่าเท่าใดในฐานสิบ

a) 11.01
b) 11.25
c) 11.5
d) 13.25

2. 0x3F7 มีค่าเท่าใดในฐานแปด

a) 1767_8
b) 3767_8
c) 1773_8
d) 7731_8

3. –100 ในระบบ 8-bit two's complement คือ

a) 0x64
b) 0xE4
c) 0x9B
d) 0x9C

4. ช่วงค่าของเลข 6-bit two's complement คือ

a) –31 ถึง 31
b) –32 ถึง 31
c) –31 ถึง 32
d) 0 ถึง 63

5. บวกเลข 8-bit two's complement 0x70 + 0x50 ข้อใดถูก

a) carry out = 1, overflow = 1
b) carry out = 0, overflow = 0
c) carry out = 0, overflow = 1
d) carry out = 1, overflow = 0

6. ข้อใดถูกต้องตามกฎ DeMorgan

a) (AB)' = A'B'
b) (A + B)' = A' + B'
c) (AB)' = A' + B'
d) (A + B)' = AB

7. F(A,B,C) = A'B'C' + A'B'C + AB'C' + AB'C ลดรูปได้เป็น

a) A'
b) B'
c) B'C'
d) A + B'

8. F(A,B,C,D) = Σm(0, 2, 8, 10) + d(1, 3, 9, 11) รูปที่ลดได้น้อยที่สุดคือ

a) B'D' b) A'B'

c) B' d) B' + D'

9. Multiplexer แบบ 32-to-1 ต้องมี select lines กี่เส้น

a) 4 b) 5

c) 16 d) 32

10. JK flip-flop ที่มี J = 1 และ K = 1 จะเกิดอะไรขึ้นที่ขอบ clock

a) Q = 0 b) Q = 1

c) Q คงเดิม d) Q กลับค่า (toggle)

11. ข้อใดถูกต้องเกี่ยวกับ Moore machine

a) output ขึ้นกับ state ปัจจุบันเท่านั้น b) output ขึ้นกับ input ปัจจุบันเท่านั้น

c) ใช้ state น้อยกว่า Mealy เสมอ d) output เขียนไว้บนเส้น transition

Computer Architecture (ข้อ 12–40)

12. Intel เพิ่ม L3 cache และ Hyper-Threading ใน CPU รุ่นใหม่ โดยโปรแกรมเดิมรันได้ทันทีไม่ต้อง compile ใหม่ นี่คือการเปลี่ยนแปลงในระดับใด

a) ISA b) ABI

c) Computer Organization d) Compiler

13. Moore's Law กล่าวว่า

a) clock rate เพิ่มเป็น 2 เท่าทุก 2 ปี b) จำนวน transistor บน IC เพิ่มเป็น 2 เท่าประมาณทุก 2 ปี (18 เดือน)

c) ราคา DRAM ลดลง 40% ต่อปี d) performance เพิ่มขึ้นตามจำนวน core

14. Computer A รันโปรแกรมใช้เวลา 15 s ส่วน Computer B ใช้ 24 s ข้อใดถูก

a) A เร็วกว่า B 1.6 เท่า b) A เร็วกว่า B 0.625 เท่า

c) B เร็วกว่า A 1.6 เท่า d) A เร็วกว่า B 9 เท่า

15. Computer X ทำ 4 งานพร้อมกันเสร็จทั้งหมดใน 8 นาที ส่วน Computer Y ทำทีละงาน งานละ 3 นาที ข้อใดถูก

a) X มี throughput สูงกว่า และ Y มี response time ต่ำกว่า b) Y ดีกว่าทั้งสองด้าน

c) X ดีกว่าทั้งสองด้าน d) throughput เท่ากัน

16. โปรแกรมมี 4×10^9 คำสั่ง รันด้วย average CPI = 1.5 บน CPU 3 GHz จะใช้ CPU time เท่าไร

a) 0.5 s b) 1.125 s

c) 2 s d) 8 s

17. ส่วนของโปรแกรมที่กินเวลา 60% ถูกทำให้เร็วขึ้น 3 เท่า overall speedup คือ

a) 1.67 b) 1.8

c) 2.0 d) 3.0

18. เหตุผลที่ควรใช้ geometric mean กับผล benchmark ที่เป็น ratio คือ

a) ได้ค่าเฉลี่ยมากที่สุด b) ให้ผลสอดคล้องกันไม่ว่าจะใช้เครื่องใดเป็น baseline

c) ทำนายเวลาจริงได้แม่นยำ d) ไม่ต้องใช้ workload

19. สถาปัตยกรรมแบบใดเป็น "0-address" (operand อยู่ที่ top of stack โดยปริยาย)

a) Accumulator b) Stack

c) Register-Memory d) Load-Store

20. เก็บค่า 0x11223344 แบบ **big endian** ที่ address 0x1000 ไบต์ที่ address **0x1003** คือ

a) 0x11 b) 0x22

c) 0x33 d) 0x44

21. คำสั่ง `LD r1, 100(r2)` ที่ใช้อ่าน field ของ struct (เช่น `A = G.age`) ใช้ addressing mode ใด

a) Register indirect b) Displacement

c) Memory indirect d) Scale

22. ภายใต้ restricted alignment address ใดเก็บ `int` (4 bytes) ได้ถูกต้อง

a) 0x1002 b) 0x1006

c) 0x100C d) 0x1011

23. ระบบใช้ **Handler Table** ที่ base 0x4000 โดย handler แต่ละตัวมีขนาด 32 bytes เมื่อเกิด interrupt หมายเลข 3 CPU จะกระโดดไปที่

a) 0x400C b) 0x4003

c) 0x4060 d) 0x4096

24. ใน nanoLADA single-cycle ค่า control ของคำสั่ง `ADD rd, rs, rt` ข้อใดถูก

a) sel_wr = 1 (rt), sel_b = 1 (imm) b) sel_wr = 0 (rd), sel_b = 0 (R[rt])

c) sel_data = 1 (memory) d) mem_wr = 1

25. ขั้นตอนแรกของการออกแบบ processor ตามสไลด์คือ

a) ประกอบ control logic b) เลือก components ของ datapath

c) วิเคราะห์ ISA โดยเขียนแต่ละคำสั่งเป็น RTL d) หา critical path

26. single-cycle nanoLADA มี critical path 230 ns clock rate สูงสุดประมาณเท่าไร

a) 230 MHz b) 43.5 MHz

c) 4.35 MHz d) 2.3 GHz

27. multi-cycle nanoLADA (cycle time 20 ns) รันโปรแกรมที่มี LW 3 คำสั่ง, ADD 2 คำสั่ง และ BEQ 1 คำสั่ง จะใช้เวลาเท่าไร

a) 120 ns
b) 480 ns
c) 520 ns
d) 600 ns

28. ข้อดีสำคัญของการ implement control ด้วย microprogram คือ

a) เร็วกว่า hardwired เสมอ
b) เปลี่ยน microprogram (Map ROM) แล้วทำ ISA อื่นบน hardware เดิมได้
c) ไม่ต้องมี state
d) ลด IC ของโปรแกรม

29. ตัวอย่าง laundry มี 4 ขั้นตอน ขั้นละ 30 นาที ถ้าทำแบบ pipeline จำนวน **6 loads** จะใช้เวลาเท่าไร

a) 3 ชั่วโมง
b) 4.5 ชั่วโมง
c) 6 ชั่วโมง
d) 12 ชั่วโมง

30. ใน pipeline ที่ LW ใช้ 5 stages และ R-type ใช้ 4 stages (ไม่มี MEM) ถ้า LW ตามด้วย ADD ทั้งสองคำสั่งจะเขียน register file ใน cycle เดียวกัน นี่คือ hazard ชนิดใด

a) Structural
b) Data (RAW)
c) Control
d) WAR

31. โค้ด `a = sin(x); x = 10;` (โดยที่ sin ต้องเตรียมนาน) อาจเกิด hazard ชนิดใด

a) RAW
b) WAW
c) WAR
d) RAR

32. ตาม Flynn's taxonomy ในสไลด์ MISD ใช้กับงานแบบใด

a) Uniprocessor
b) Vector processor
c) Fault-tolerant computing
d) Multicore

33. Superpipeline ที่มี m = 3 minor cycles, k = 5 stages รัน n = 301 คำสั่ง (ideal) ใช้เวลาเท่าไร

a) 105 T_c
b) 305 T_c
c) 101 T_c
d) 903 T_c

34. ข้อใด**ไม่ใช่**ข้อจำกัดหรือข้อเสียของ loop unrolling

a) code size ใหญ่ขึ้น
b) จำนวน register จำกัดจำนวนรอบที่ unroll ได้
c) ทำให้ basic block เล็กลง
d) ต้อง rename register เพื่อให้แต่ละ iteration อิสระกัน

35. Fully associative cache ยังคงมี miss ชนิดใดบ้าง

a) Conflict อย่างเดียว
b) Compulsory และ Capacity
c) Compulsory, Capacity และ Conflict
d) ไม่มี miss เลย

36. Direct-mapped cache ขนาด 64 bytes มี block size 8 bytes address 300 (ฐานสิบ) จะอยู่ที่ index ใด

a) 4
b) 5
c) 37
d) 44

37. Write-through cache ที่ write buffer เต็มบ่อย (saturate) เพราะ memory ช้ามาก วิธีแก้ที่สไลด์เสนอคือ

a) เพิ่มขนาด write buffer ไปเรื่อย ๆ
b) เพิ่ม L2 cache
c) ปิด cache
d) ใช้ direct-mapped

38. TLB มักจัดเป็น fully associative เพราะ

a) ขนาดใหญ่มาก
b) มีขนาดเล็ก (ประมาณ 128–256 entries) จึงเทียบทุก entry ได้
c) ไม่มี tag
d) OS จัดการทั้งหมด

39. Virtual address 32 bits, page size 8 KB และ page table entry 4 bytes ขนาด page table ต่อ process คือ

a) 1 MB
b) 2 MB
c) 4 MB
d) 8 MB

40. RAID 5 ประกอบด้วยดิสก์ 2 TB จำนวน 4 ลูก จะมีพื้นที่ใช้งานจริงเท่าไร

a) 2 TB
b) 4 TB
c) 6 TB
d) 8 TB

Part 2: ตอบสั้น (ข้อละ 2 คะแนน)

ตอบเป็นตัวเลขพร้อมหน่วย หรือวลีสั้น ๆ

Digital Logic

S1. เขียนสมการ Boolean ของ **Sum** และ **Carry-out** ของ full adder (อินพุต A, B, Cin)

S2. ออกแบบ **Moore machine** ตรวจจับลำดับ "101" แบบ overlapping จะต้องใช้กี่ state และต้องใช้ flip-flop **อย่างน้อย** กี่ตัว (binary encoding)

Computer Architecture

S3. โปรแกรมมี 2×10^9 คำสั่ง รันบน CPU 2 GHz ด้วย instruction mix ดังนี้: ALU 40% (CPI 1), Load 30% (CPI 4), Store 10% (CPI 3), Branch 20% (CPI 2) จงหา **average CPI** และ **CPU time**

S4. ส่วนที่ปรับปรุงได้กินเวลา 90% ของโปรแกรม ต้องทำให้ส่วนนั้นเร็วขึ้นกี่เท่า จึงจะได้ overall speedup = 5

S5. ภายใต้ restricted alignment ตัวแปรถูกวางตามลำดับนี้โดยเริ่มที่ address 0: `char a; int b; short c; double d; char e;` ตัวแปร **d** เริ่มที่ address ใด และ**ไบต์สุดท้ายที่ใช้**คือ address ใด

S6. Single-cycle processor มี cycle time 200 ns ส่วน multi-cycle (nanoLADA: LW 5, SW 4, R-type/ORI 4, BEQ 3 cycles) มี cycle time 50 ns ถ้า instruction mix คือ LW 20%, SW 10%, R-type/ORI 50% และ BEQ 20% จงหา **average CPI ของ multi-cycle** และบอกว่าเครื่องไหนเร็วกว่า

S7. พิจารณา 5-stage pipeline ที่มี full forwarding ระบุ data dependency ทั้งหมด (ชนิดและ register) และบอกว่าต้อง stall กี่ cycle

```
1: LW  $r1, 0($r2)
2: ADD $r3, $r1, $r1
3: SW  $r3, 4($r2)
4: ADD $r1, $r4, $r5
```

S8. ทำไม RAID 5 จึง**เขียน**ได้เร็วกว่า RAID 4 (ตอบ 1–2 ประโยค) และ RAID 5 ต้องใช้ดิสก์อย่างน้อยกี่ลูก

S9. ระบบมี address 32 bits ใช้ **4-way set associative cache ขนาด 64 KB** โดย **block size 64 bytes** จงหาจำนวนบิตของ **tag, index และ offset**

S10. ระบบ cache 2 ระดับ: L1 hit time 1 cycle, L1 miss rate 4%, L2 hit time 12 cycles, L2 local miss rate 25% และ memory access 200 cycles จงหา **AMAT**

ชุดที่ 2: ฝึกเพิ่มแบบข้อเขียนยาว (100 คะแนน, 90 นาที)

ชุดนี้เป็น Computer Architecture อย่างเดียว เน้นการคำนวณและอธิบายหลายขั้นตอน เพื่อให้ทำข้อตอบสั้นในชุดที่ 1 ได้คล่อง

ส่วน	รูปแบบ	คะแนน	เวลาแนะนำ
A	เติมคำ 20 ข้อ	20	15 นาที
B	ปรนัย 20 ข้อ	20	20 นาที
C	คำนวณและอธิบาย 8 ข้อ	60	55 นาที

Part A: เติมคำ (ข้อละ 1 คะแนน)

1. ISA เป็น abstraction/interface ระหว่าง _____ กับ _____
2. ในปี 1964 IBM ออกสถาปัตยกรรม _____ ซึ่งรวม instruction set ที่ไม่เข้ากันได้ 7 แบบเข้าด้วยกัน
3. Core i3, i5 และ i7 ใช้ ISA เดียวกัน ความแตกต่างของมันจึงอยู่ที่ระดับ _____
4. Benchmark ที่พยายามเลียนแบบพฤติกรรมของโปรแกรมจริง เช่น Whetstone และ Dhrystone เรียกว่า _____ benchmark
5. เมื่อต้องสรุปผล benchmark ที่เป็น **ratio** ควรใช้ค่าเฉลี่ยแบบ _____ mean
6. เก็บค่า `0xCAFE1234` แบบ **little endian** ที่ address 0x200 ไบต์ที่ address **0x201** คือ `0x____`
7. ภายใต้ restricted alignment ข้อมูลขนาด s ไบต์ต้องมี base address A ที่ทำให้ _____ = 0
8. Addressing mode ของ `LD r1, (r2)+` ชื่อ _____ ซึ่งตรงกับ C ว่า `A = D[____]`
9. ใน nanoLADA คำสั่ง I-type มี immediate __ bits และ J-type มี address field __ bits
10. คำสั่ง `ORUI` ใช้ extender แบบ _____ ส่วน `LW` / `SW` / `BEQ` ใช้แบบ _____
11. ใน single-cycle nanoLADA คำสั่งที่กำหนด critical path คือ _____
12. ใน multi-cycle nanoLADA คำสั่ง LW, SW และ JMP ใช้ __, __ และ __ cycles ตามลำดับ
13. Control unit ของ multi-cycle processor เป็น _____ ซึ่งสร้างได้ด้วย sequential circuit หรือ _____
14. ถ้า Wregs(i) ∩ Rregs(j) ≠ ∅ โดย j มาก่อน i จะเกิด data hazard ชนิด _____
15. ตาม Flynn's taxonomy, vector processor/GPU คือ _____ และ multicore คือ _____
16. Simultaneous Multithreading (SMT) ของ Intel มีชื่อทางการค้าว่า _____
17. _____ ใช้ capacitor ต้อง refresh และเหมาะเป็น main memory ส่วน _____ ใช้ transistor และเหมาะเป็น cache
18. Miss ที่เกิดขึ้นทั้งที่ cache ยังมีที่ว่าง แต่ mapping บังคับให้ชนกัน เรียกว่า _____ miss
19. Miss ใน virtual memory เรียกว่า _____ และใช้นโยบายการเขียนแบบ write-_____ เท่านั้น
20. RAID __ ใช้ parity สองชุดและทนดิสก์เสียได้ 2 ลูก ส่วน RAID 5 ต้องใช้ดิสก์อย่างน้อย __ ลูก

Part B: ปรนัย (ข้อละ 1 คะแนน)

B1. ข้อใด**ไม่ใช่**สิ่งที่ ISA กำหนด

a) รูปแบบ (format) ของคำสั่ง
b) addressing modes
c) จำนวน cache level ใน CPU
d) วิธีจัดการ interrupt/exception

B2. Computer A มี 400 MIPS และรันโปรแกรมที่ compile ได้ 800 ล้านคำสั่ง ส่วน Computer B (ISA ต่างกัน) มี 300 MIPS และรันโปรแกรมเดียวกันที่ compile ได้ 500 ล้านคำสั่ง ข้อใดถูก

a) A เร็วกว่าเพราะ MIPS สูงกว่า
b) B เร็วกว่าประมาณ 1.2 เท่า
c) เร็วเท่ากัน
d) สรุปไม่ได้เพราะไม่รู้ clock rate

B3. สถาปัตยกรรมใดเหมาะกับ pipelining ที่สุด

a) Accumulator
b) Stack
c) Memory-Memory
d) Register-Register (Load-Store)

B4. ข้อใดเป็นข้อเสียของ Stack architecture ตามสไลด์

a) code density แย่
b) ข้อมูลที่ต้องการไม่ได้อยู่ top of stack เสมอ
c) ต้องใช้ hardware มาก
d) compiler เขียนยาก

B5. สำหรับการ**เปรียบเทียบ (มากกว่า/น้อยกว่า)** เลข 32 บิตบน bus 8 บิต endian แบบใดเร็วกว่า และเพราะอะไร

a) Little: ได้ byte ต่ำก่อน
b) Big: ได้ byte ที่มี sign bit ก่อน
c) Little: carry ถูกต้อง
d) เท่ากัน

B6. ระบบใช้ **Vector Table** ที่ base 0x8000 และ address ขนาด 4 bytes เมื่อเกิด interrupt หมายเลข 5 CPU จะอ่าน**ที่อยู่ของ handler** จาก address ใด

a) 0x8005
b) 0x8014
c) 0x8050
d) 0x8020

B7. วิธี clear register ที่สไลด์แนะนำสำหรับ architecture ส่วนใหญ่คือ

a) `CLR R1`
b) `MOV R1, 0`
c) `SUB R1,R1,R1`
d) `XOR R1,R1,R1`

B8. การย้าย `hs = 3600;` ที่เป็นค่าคงที่ออกไปไว้นอก loop เป็น optimization ประเภทใด

a) Strength reduction
b) Code motion
c) Procedure inlining
d) Pipeline scheduling

B9. ใน nanoLADA single-cycle คำสั่งใดมี `reg_wr = 0` และ `mem_wr = 1`

a) LW
b) SW
c) BEQ
d) ORI

B10. หลังรัน `ORUI r1, r0, #0xBEEF` ค่าของ r1 คือ

a) 0x0000BEEF
b) 0xFFFFBEEF
c) 0xBEEF0000
d) 0xBEEFBEEF

B11. คำสั่ง `BEQ r1, r2, imm` อยู่ที่ PC = 0x0040 โดย imm = −3 และ R[r1] = R[r2] ค่า PC ถัดไปคือ

a) 0x0034
b) 0x0038
c) 0x003D
d) 0x0044

B12. Single-cycle มี cycle time 240 ns ส่วน multi-cycle มี cycle time 60 ns ถ้าต้องการให้ multi-cycle เร็ว**อย่างน้อยเท่ากับ** single-cycle ค่า average CPI ต้องเป็นเท่าไร

a) ≥ 4
b) ≤ 4
c) ≤ 5
d) = 1

B13. ข้อใด**ถูก**เกี่ยวกับ pipelining

a) ลด latency ของแต่ละคำสั่ง
b) เพิ่ม throughput โดย clock ถูกจำกัดด้วย stage ที่ช้าที่สุด
c) speedup เท่ากับจำนวน stage พอดีเสมอ
d) stage ที่ไม่สมดุลไม่มีผล

B14. การแยก instruction cache กับ data cache ช่วยแก้ hazard ชนิดใด

a) Structural
b) Data (RAW)
c) Control
d) WAW

B15. ใน 5-stage pipeline ที่มี forwarding ครบ ลำดับคำสั่งใด**ที่ยังต้อง stall**

a) `ADD r1,..` ตามด้วย `SUB ..,r1,..`
b) `LW r1,0(r2)` ตามด้วย `ADD r3,r1,r4`
c) `SW r1,0(r2)` ตามด้วย `ADD r3,r4,r5`
d) `ADD r1,..` ตามด้วย `SW r1,0(r2)`

B16. ข้อใดอธิบาย **branch delay slot** ได้ถูกต้อง

a) hardware ยกเลิกคำสั่งหลัง branch ทุกครั้ง
b) คำสั่งถัดจาก branch จะถูก execute เสมอ ไม่ว่า branch จะ taken หรือไม่
c) เป็นวิธีทาย branch ด้วย history
d) ไม่ต้องอาศัย compiler

B17. Superscalar degree 2 กับ pipeline 5 stage รัน 1000 คำสั่ง (ideal) ใช้เวลาประมาณกี่ T_C

a) 1004
b) 504
c) 502
d) 1000

B18. ข้อใด**ถูก**เกี่ยวกับ VLIW

a) ใช้ dynamic scheduling ใน hardware
b) ลด CPI แต่ไม่ลด IC
c) ใช้ static scheduling ทำให้ binary ข้ามรุ่น (Itanium → Itanium II) ไม่ compatible
d) Intel Itanium ยังผลิตอยู่และได้รับความนิยม

B19. การเพิ่ม associativity จาก direct-mapped เป็น 4-way (cache size เท่าเดิม) มีผลข้อใดมากที่สุด

a) ลด compulsory miss
b) ลด conflict miss แต่ hit time อาจเพิ่มขึ้น
c) ลด capacity miss เป็นศูนย์
d) ทำให้ไม่ต้องมี tag

B20. Amazon EBS และ NAS ที่ใช้ NFS ให้บริการ storage ระดับใดตามลำดับ

a) Block, File
b) File, Block
c) Object, File
d) Block, Object

Part C: คำนวณและอธิบาย (60 คะแนน) แสดงวิธีทำ

C1. CPU Performance (8 คะแนน)

โปรแกรมหนึ่งมี 10^9 คำสั่ง รันบน CPU 2.5 GHz โดยมี instruction mix ดังนี้

Class	สัดส่วน	CPI
ALU	50%	1
Load	20%	5
Store	10%	4
Branch	20%	3

- (a) หา average CPI และ CPU time (3 คะแนน)
- (b) ถ้าปรับปรุง memory จน CPI ของ Load ลดเหลือ 3 จะได้ speedup เท่าไร (2 คะแนน)
- (c) แสดงว่าคำตอบข้อ (b) ได้ค่าเท่ากันเมื่อใช้ Amdahl's Law และอธิบายว่า P ที่ต้องใช้คืออะไร ทำไม**ไม่ใช่** 20% (3 คะแนน)

C2. Amdahl's Law (5 คะแนน)

โปรแกรมหนึ่งใช้เวลา 70% กับส่วนที่ทำ parallel ได้

- (a) ถ้ารันบน 8 cores จะได้ overall speedup เท่าไร (2 คะแนน)
- (b) speedup สูงสุดที่เป็นไปได้ (cores $\to \infty$) คือเท่าไร (1 คะแนน)
- (c) ทีมงานมีสองทางเลือก คือ (i) เพิ่ม cores จาก 8 เป็น 64 หรือ (ii) ทำให้ส่วน serial เร็วขึ้น 2 เท่า (ยังคง 8 cores) ทางไหนดีกว่า แสดงตัวเลข (2 คะแนน)

C3. Single-cycle vs Multi-cycle vs Pipeline (9 คะแนน)

พิจารณาโปรแกรมต่อไปนี้ (BEQ **not taken**) โดย single-cycle มี cycle time 230 ns, multi-cycle มี cycle time 50 ns และ pipeline 5 stage มี cycle time 50 ns (ideal ไม่มี stall)

```
LW   $r1, 0($r10)
LW   $r2, 4($r10)
ADD  $r3, $r1, $r2
ORI  $r4, $r3, #1
SW   $r4, 8($r10)
BEQ  $r1, $r2, SKIP
JMP  END
```

- (a) หาเวลาที่ใช้บน single-cycle (2 คะแนน)
- (b) หาจำนวน cycle และเวลาที่ใช้บน multi-cycle (3 คะแนน)
- (c) หาเวลาที่ใช้บน ideal pipeline และ speedup เทียบกับ single-cycle (2 คะแนน)
- (d) ในความเป็นจริง pipeline นี้จะเจอ hazard อย่างน้อย 2 จุด ให้ระบุว่าอยู่ระหว่างคำสั่งใดและเป็นชนิดใด (2 คะแนน)

C4. Cache Address Mapping (10 คะแนน)

ระบบมี address 32 bits และ **direct-mapped cache ขนาด 16 KB** โดย **block size 32 bytes**

- (a) หาจำนวนบิตของ tag, index และ offset (3 คะแนน)
- (b) หา tag, index และ offset (ฐานสิบ) ของ address `0x00012345` (3 คะแนน)
- (c) address `0x00016345` จะ conflict กับ `0x00012345` หรือไม่ อธิบาย (2 คะแนน)
- (d) ถ้าเปลี่ยนเป็น **2-way set associative** ขนาดเท่าเดิม จำนวนบิตของ tag/index/offset จะเป็นเท่าไร และต้องใช้ comparator กี่ตัว (2 คะแนน)

C5. Cache Performance (6 คะแนน)

- (a) CPU มี CPI_{ideal} = 1 และ 30% ของคำสั่งเป็น load/store โดย I-cache miss rate = 2%, D-cache miss rate = 5% และ miss penalty = 100 cycles จงหา CPI จริง และบอกว่าช้ากว่า CPU ที่มี cache สมบูรณ์แบบกี่เท่า (3 คะแนน)
- (b) ระบบ 2-level มี L1 hit time 1 cycle, L1 miss rate 5%, L2 hit time 10 cycles, L2 local miss rate 20% และ memory 100 cycles จงหา AMAT (3 คะแนน)

C6. Pipeline Hazards & Scheduling (10 คะแนน)

พิจารณา 5-stage pipeline ที่มี **full forwarding** โดย register file เขียนครึ่งแรกของ cycle และอ่านครึ่งหลัง

```
1: LW   $r1, 0($r2)
2: ADD  $r3, $r1, $r4
3: LW   $r6, 4($r2)
4: SUB  $r7, $r6, $r3
5: ORI  $r8, $r9, #5
```

- (a) ระบุ data dependency ทั้งหมด (คู่คำสั่ง, register และชนิด RAW/WAR/WAW) (3 คะแนน)
- (b) dependency ใดบ้างที่ forwarding แก้ได้โดยไม่ต้อง stall และจุดใดที่ยังต้อง stall รวมแล้วโปรแกรมนี้ใช้กี่ cycle (3 คะแนน)
- (c) **Reschedule** โปรแกรมให้ไม่มี stall เลย (ผลลัพธ์ต้องถูกต้องเหมือนเดิม) และบอกจำนวน cycle ใหม่ (3 คะแนน)
- (d) ถ้ารันบน machine ที่**ไม่มี forwarding เลย** software จะแก้ hazard ที่เหลือโดยไม่ต้องให้ hardware detect ได้อย่างไร (1 คะแนน)

C7. Virtual Memory & TLB (6 คะแนน)

ระบบมี virtual address 32 bits, page size 4 KB, physical memory 1 GB และ page table entry ขนาด 4 bytes

- (a) หาจำนวนบิตของ VPN, page offset และ PPN (2 คะแนน)
- (b) page table ของแต่ละ process มีขนาดเท่าไร (1 คะแนน)
- (c) สถาปนิกต้องการ **L1 cache ขนาด 32 KB** ที่ทำ **overlapped cache & TLB access** ได้ cache ต้องมี associativity อย่างน้อยเท่าไร อธิบาย (2 คะแนน)
- (d) เมื่อ OS ทำ process switch ต้องทำอะไรกับ TLB และ page table (1 คะแนน)

C8. อธิบายสั้น ๆ: เลือกตอบ 2 ข้อจาก 4 ข้อ (ข้อละ 3 คะแนน)

- (a) ทำไม **RAID 5** จึงเขียนได้เร็วกว่า **RAID 4** และ RAID 0 เหมาะกับงานแบบใด
- (b) เปรียบเทียบ **write-back** กับ **write-through** ในแง่ความเร็วและการใช้งานกับ multiprocessor และอธิบายว่า write buffer มีไว้ทำอะไร และเมื่อไรที่มันจะ saturate
- (c) โค้ดสองแบบนี้ให้ผลเหมือนกัน แบบไหนเร็วกว่า เพราะอะไร

```
int data[1000][1000];
// แบบ X                                  // แบบ Y
for (r=0;r<1000;r++)                      for (c=0;c<1000;c++)
  for (c=0;c<1000;c++) sum+=data[r][c];     for (r=0;r<1000;r++) sum+=data[r][c];
```

- (d) เปรียบเทียบ **dynamic scheduling** (superscalar OoO) กับ **static scheduling** (VLIW) ในแง่ hardware, compatibility และ performance ของ legacy software แล้วบอกว่าในทางปฏิบัติฝั่งไหน "ชนะ" เพราะอะไร

เฉลยละเอียด ชุดที่ 1

ตารางคำตอบ (ตรวจเร็ว)

ข้อ	1	2	3	4	5	6	7	8	9	10
ตอบ	b	a	d	b	c	c	b	c	b	d
ข้อ	**11**	**12**	**13**	**14**	**15**	**16**	**17**	**18**	**19**	**20**
ตอบ	a	c	b	a	a	c	a	b	b	d
ข้อ	**21**	**22**	**23**	**24**	**25**	**26**	**27**	**28**	**29**	**30**
ตอบ	b	c	c	b	c	c	c	b	b	a
ข้อ	**31**	**32**	**33**	**34**	**35**	**36**	**37**	**38**	**39**	**40**
ตอบ	c	c	a	c	b	b	b	b	b	c

แยกคะแนน: Digital Logic = ข้อ 1–11 + S1–S2 (เต็ม 15) · Computer Architecture = ข้อ 12–40 + S3–S10 (เต็ม 45)

Part 1: Choice (อธิบายรายข้อ)

Digital Logic

1. ตอบ b) 11.25

$(1011.01)_2 = 1{\cdot}8 + 0{\cdot}4 + 1{\cdot}2 + 1{\cdot}1 + 0{\cdot}½ + 1{\cdot}¼ = 8 + 2 + 1 + 0.25 =$ **11.25**

- **ตัวลวง a:** อ่านตัวเลขหลังจุดเป็นฐานสิบตรง ๆ
- **ตัวลวง c:** คิดบิตหลังจุดผิดตำแหน่ง ($0.1_2 = 0.5$ แต่ $0.01_2 = 0.25$)
- **หลักที่ใช้:** หลังจุดทศนิยม ค่าประจำหลักคือ 2^{-1}, 2^{-2}, ...

2. ตอบ a) 1767_8

0x3F7 = 0011 1111 0111 (หลักละ 4 บิต) แล้วจัดใหม่ทีละ 3 บิตจากขวาเป็น 001 111 110 111 = 1 7 6 7

- **ตรวจ:** $1{\cdot}512 + 7{\cdot}64 + 6{\cdot}8 + 7 = 1015$ และ $0x3F7 = 3{\cdot}256 + 15{\cdot}16 + 7 = 1015$ ✓
- **ตัวลวง b:** แปลง "3" เป็นหลักแรกตรง ๆ (ลืมว่าต้องจัดกลุ่มใหม่ทั้งหมด)

3. ตอบ d) 0x9C

1. +100 = 0110 0100
2. กลับทุกบิตได้ 1001 1011 (= 0x9B ซึ่งเป็น **1's complement**: ตัวลวง c)
3. บวก 1 ได้ **1001 1100 = 0x9C**

- **ตรวจ:** 0x9C = 156 และ 156 − 256 = −100 ✓
- **ตัวลวง b (0xE4):** แค่เปลี่ยน sign bit แบบ sign-magnitude

4. ตอบ b) −32 ถึง 31

2's complement n bits มีช่วง -2^{n-1} ถึง $2^{n-1} - 1$ → n = 6: **−32 ถึง 31**

- **ตัวลวง a:** เป็นช่วงของ sign-magnitude และ 1's complement
- **ตัวลวง d:** เป็นช่วงของ unsigned
- ช่วงของ 2's complement **ไม่สมมาตร** (ฝั่งลบมีมากกว่า 1 ค่า เพราะมีศูนย์ตัวเดียว)

5. ตอบ c) carry out = 0, overflow = 1

0111 0000 + 0101 0000 = **1100 0000** (ไม่มีบิตทดออกจาก MSB → carry = 0)

- แต่ **บวก + บวก ได้ผลที่ MSB = 1 (ลบ) → signed overflow** (112 + 80 = 192 เกิน 127)
- ตรวจด้วยสูตร overflow = $C_{in,MSB} \oplus C_{out,MSB} = 1 \oplus 0 = 1$
- **หลัก:** carry มีความหมายกับ unsigned ส่วน overflow มีความหมายกับ signed ทั้งสองเป็นอิสระต่อกัน

6. ตอบ c) (AB)' = A' + B'

DeMorgan: กลับทุกตัวแปรแล้วสลับ AND ↔ OR → (AB)' = A' + B' และ (A + B)' = A'B'

- ตัวเลือก a, b เป็นความผิดพลาดที่พบบ่อย (กลับตัวแปรแต่ลืมสลับเครื่องหมาย)
- ตัวเลือก d ลืมกลับตัวแปร

7. ตอบ b) B'

พจน์ทั้ง 4 คือ minterm 000, 001, 100, 101 = m(0,1,4,5) ซึ่ง**ทุกตัวมี B = 0** ส่วน A และ C เปลี่ยนครบทุกค่า

- จัดกลุ่มเชิงพีชคณิต: A'B'(C' + C) + AB'(C' + C) = A'B' + AB' = B'(A' + A) = **B'**
- ใน K-map ทั้ง 4 ช่องอยู่ในคอลัมน์ B = 0 เป็นกลุ่มเดียว

8. ตอบ c) B'

- m(0,2,8,10) อยู่ที่ **มุมทั้งสี่** ของ K-map (A'B'C'D', A'B'CD', AB'C'D', AB'CD') ถ้าไม่มี don't care จะได้ B'D' (ตัวลวง a ซึ่งถูกแต่ยังไม่ minimal)
- เมื่อนับ d(1,3,9,11) เป็น 1 จะได้ 8 ช่องที่ B = 0 ทั้งหมด (แถว AB = 00 และ 10) → กลุ่ม 8 ช่อง = **B'** ซึ่งสั้นที่สุด
- **หลัก:** ใช้ don't care **เมื่อทำให้กลุ่มใหญ่ขึ้น**

9. ตอบ b) 5

MUX 2^n-to-1 มี select n เส้น → 32 = 2^5 ได้ **5 เส้น**

- **ตัวลวง d (32):** นับจำนวน data input แทน select

10. ตอบ d) toggle

$Q^+ = JQ' + K'Q$ → J = K = 1 ได้ $Q^+ = Q'$ (**กลับค่า**)

- ตารางของ JK: 00 = hold, 01 = reset, 10 = set, **11 = toggle**
- **กับดัก:** สถานะต้องห้าม (S = R = 1) เป็นของ **SR latch** ไม่ใช่ JK

11. ตอบ a)

Moore: output ขึ้นกับ **current state อย่างเดียว** (เขียน output ไว้ในวงกลม state)

- **ตัวลวง d:** output บนเส้น transition เป็นของ **Mealy**
- **ตัวลวง c:** กลับกัน คือ Moore มักใช้ state **มากกว่า** Mealy

Computer Architecture

12. ตอบ c) Computer Organization

binary เดิมรันได้โดยไม่ต้อง compile ใหม่ แสดงว่า ISA ไม่เปลี่ยน ส่วน cache และ Hyper-Threading เป็นโครงสร้างภายใน (organization)

- ตรงกับตัวอย่างในสไลด์: **Core i3/i5/i7 ต่างกันที่ organization**
- **ตัวลวง a:** ถ้าเปลี่ยน ISA โปรแกรมเดิมอาจรันไม่ได้

13. ตอบ b)

Moore's Law เป็น**ข้อสังเกต**ของ Gordon Moore เรื่อง**จำนวน transistor** ที่เพิ่มขึ้นเป็น 2 เท่าทุกประมาณ 2 ปี (18 เดือน)

- **ตัวลวง a:** Moore's Law ไม่ได้พูดถึง clock rate
- **ตัวลวง c:** ราคา DRAM ลด 40% ต่อปีเป็น trend อีกเรื่องในสไลด์

14. ตอบ a) A เร็วกว่า B 1.6 เท่า

Speedup = $T_{ช้า} / T_{เร็ว}$ = 24 / 15 = **1.6**

- **ตัวลวง b:** 15/24 = 0.625 เป็นการกลับเศษส่วน
- **ตัวลวง d:** 24 – 15 = 9 เป็นผลต่าง ไม่ใช่อัตราส่วน

15. ตอบ a)

- **Throughput:** X = 4 งาน / 8 นาที = **0.5 งาน/นาที** ส่วน Y = 1/3 = **0.33 งาน/นาที** → X ดีกว่า
- **Response time:** X ใช้ 8 นาทีต่องาน ส่วน Y ใช้ **3 นาที** → Y ดีกว่า
- **หลัก:** throughput กับ response time เป็นคนละมิติกัน ระบบที่ดีด้านหนึ่งไม่จำเป็นต้องดีอีกด้าน (ตัวอย่างเดียวกับในสไลด์)

16. ตอบ c) 2 s

CPU time = IC × CPI / f = $(4 \times 10^9 \times 1.5) / (3 \times 10^9)$ = 6/3 = **2 s**

- **ตัวลวง b:** คูณ clock แทนที่จะหาร

17. ตอบ a) 1.67

Amdahl: 1 / ((1 − 0.6) + 0.6/3) = 1 / (0.4 + 0.2) = 1/0.6 = **1.67**

- **ตัวลวง d (3.0):** คิดว่าทั้งโปรแกรมเร็วขึ้นเท่าส่วนที่เร่ง ซึ่งลืมส่วน 40% ที่ไม่ได้เร่ง

18. ตอบ b)

GM ของ ratio **ให้ผลการจัดอันดับเหมือนกันไม่ว่าจะเลือกเครื่องใดเป็น baseline** (ในสไลด์ GM(A/B) / GM(A/C) = GM(C/B))

- **ตัวลวง c:** สไลด์บอกชัดว่า **GM ทำนาย performance จริงไม่ได้**

19. ตอบ b) Stack

- Stack = **0-address** (ใช้ top of stack)
- Accumulator = 1-address
- Register-Memory = 2-address
- Load-Store = 3-address

20. ตอบ d) 0x44

Big endian เก็บ **MSB ที่ address ต่ำ**: 0x1000 = 11, 0x1001 = 22, 0x1002 = 33, **0x1003 = 44**

- **ตัวลวง a (0x11):** เป็นคำตอบถ้าเป็น little endian

21. ตอบ b) Displacement

Displacement = M[R + ค่าคงที่] ตรงกับ **field ใน struct** (R = base address ของ struct และค่าคงที่ = offset ของ field) และตัวแปร local บน stack

- Register indirect = `*p`
- Memory indirect = `**p`
- Scale = `E[i]` (R2 + R3 × 4)

22. ตอบ c) 0x100C

Restricted alignment ต้องมี address mod 4 = 0 → 0x100C = ...1100 ซึ่งสองบิตล่างเป็น 00 ✓

- 0x1002 mod 4 = 2, 0x1006 mod 4 = 2 และ 0x1011 mod 4 = 1 → ผิดทั้งหมด

23. ตอบ c) 0x4060

Handler table: address = Base + n × handler size = 0x4000 + 3 × 32 = 0x4000 + 96 = 0x4000 + **0x60**

- **ตัวลวง a (0x400C):** คิดเป็น vector table (3 × 4 bytes)
- **ตัวลวง d:** ใช้ 96 เป็น hex ทั้งที่เป็นฐานสิบ
- **หลัก:** handler table เก็บ**โค้ด** จึงคูณด้วย**ขนาด handler** ส่วน vector table เก็บ**address** จึงคูณด้วย**ขนาด address**

24. ตอบ b)

ADD เป็น R-type: **เขียนลง rd** (sel_wr = 0) และ **operand B มาจาก R[rt]** (sel_b = 0) รวมทั้ง sel_data = 0 (ผล ALU), reg_wr = 1, mem_wr = 0, alu_ops = 00

- **ตัวลวง a:** เป็นค่าของ I-type เช่น ORI หรือ LW

25. ตอบ c)

Design steps:

1. **Analyze ISA → RTL**
2. select components
3. assemble datapath
4. analyze control points
5. assemble control logic

ต้องรู้ก่อนว่าแต่ละคำสั่งทำอะไร (RTL) จึงจะรู้ว่า datapath ต้องมีอะไร

26. ตอบ c) 4.35 MHz

f_{max} = 1 / 230 ns = 1 / (230 × 10^{-9}) ≈ 4.35 × 10^{6} Hz

- **ตัวลวง a:** คิดว่า 230 ns ตรงกับ 230 MHz
- **หลัก:** T_c = critical path ดังนั้น f = 1/T_c

27. ตอบ c) 520 ns

LW 5 × 3 = 15, ADD 4 × 2 = 8, BEQ 3 × 1 = 3 → รวม **26 cycles** × 20 ns = **520 ns**

- **ตัวลวง a (120 ns):** นับแบบ single cycle (6 คำสั่ง × 20)
- **ตัวลวง d (600 ns):** คิดว่าทุกคำสั่งใช้ 5 cycles

28. ตอบ b)

Microprogram เก็บ control เป็นข้อมูลใน ROM **เปลี่ยน Map ROM แล้วทำ ISA อื่นบน HW เดิมได้**

- **ตัวลวง a:** hardwired control มักเร็วกว่า microprogram
- **ตัวลวง d:** microprogram ไม่ได้ลด IC (IC ขึ้นกับ ISA และ compiler)

29. ตอบ b) 4.5 ชั่วโมง

load แรกต้องผ่านครบ 4 ขั้น = 2 h แล้ว load ถัดไปเสร็จทุก 30 นาที → 2 + 5 × 0.5 = **4.5 h** (สูตร (k + 3)/2 กับ k = 6)

- **ตัวลวง d:** 6 × 4 × 0.5 = 12 h เป็นแบบ sequential

30. ตอบ a) Structural

ทั้งสองคำสั่ง**แย่ง resource เดียวกัน** (write port ของ register file มีเพียง 1 port) ใน cycle เดียวกัน

- **ไม่ใช่ data hazard** เพราะสองคำสั่งนี้อาจไม่มี dependency กันเลยก็ได้
- **วิธีแก้ในสไลด์:** ใส่ bubble หรือให้ R-type ใช้ 5 stages (MEM เป็น NOOP)

31. ตอบ c) WAR

คำสั่งแรก**อ่าน** x และคำสั่งหลัง**เขียน** x ถ้าการเขียนเกิดก่อนการอ่าน (เพราะ sin เตรียมนาน) คำสั่งแรกจะได้ค่า x ที่ผิด → **Write After Read**

- **ตัวลวง d:** RAR ไม่มีปัญหา จึงไม่นับเป็น hazard

32. ตอบ c) Fault-tolerant computing

- MISD = หลาย instruction stream ทำกับข้อมูลชุดเดียว เช่นคำนวณซ้ำหลายวิธีแล้วเทียบผล → **fault tolerance**
- SISD = uniprocessor
- SIMD = vector processor
- MIMD = multicore

33. ตอบ a) 105 T_c

Superpipeline: $T = (k + (n - 1)/m)\ T_c = 5 + 300/3 = 5 + 100 =$ **105 T_c**

- **ตัวลวง b (305):** ใช้สูตร pipeline ธรรมดา (5 + 300)
- **หลัก:** หลังเติม pipe แล้ว คำสั่งเสร็จทุก minor cycle (1/m ของ T_c)

34. ตอบ c)

Loop unrolling ทำให้ basic block **ใหญ่ขึ้น** ซึ่งเป็นจุดประสงค์ของมัน (มีคำสั่งอิสระมาเติม slot มากขึ้น) ข้อ c จึงไม่ใช่ข้อเสีย

- a (code ใหญ่ขึ้น) และ b (register จำกัดจำนวนรอบ) เป็นข้อเสียจริงตามสไลด์
- d (ต้อง rename register) เป็นต้นทุนที่ต้องทำเพิ่ม

35. ตอบ b) Compulsory และ Capacity

Fully associative วาง block ได้ทุกที่ จึง**ไม่มี conflict miss** แต่การเข้าถึงครั้งแรกยัง miss (compulsory) และถ้า working set ใหญ่กว่า cache ก็ยัง miss (capacity)

36. ตอบ b) 5

Index = (300 mod 64) div 8 = 44 div 8 = **5** (tag = 300 div 64 = 4 และ offset = 44 mod 8 = 4)

- **ตัวลวง d (44):** ลืมหารด้วย block size
- **ตัวลวง c (37):** ใช้ 300 div 8 โดยลืม mod cache size
- **ตัวลวง a (4):** เป็นค่าของ tag (หรือ offset)

37. ตอบ b) เพิ่ม L2 cache

สไลด์บอกว่า write buffer saturate **"regardless the size of write buffer"** ถ้า memory ช้ากว่าอัตราการเขียน ทางแก้คือ **L2 cache** ("วิ่งไปแค่โถงทางเดิน แทนการวิ่งไปหอประชุม")

- **ตัวลวง a:** สไลด์บอกตรง ๆ ว่าเพิ่มขนาดไม่ช่วย

38. ตอบ b)

TLB มีขนาดเล็ก (ไม่เกิน 128–256 entries) จึงสร้าง comparator ครบทุก entry ได้ → **fully associative** (ไม่มี conflict miss)

39. ตอบ b) 2 MB

page 8 KB = 2^{13} → offset 13 bits → VPN = 32 − 13 = 19 bits → entries = 2^{19} → ขนาด = $2^{19} \times 4$ B = 2^{21} B = **2 MB**

- **ตัวลวง c (4 MB):** ใช้ page 4 KB (12 bits) ตามตัวอย่างในสไลด์

40. ตอบ c) 6 TB

RAID 5 เสียพื้นที่ให้ parity เท่ากับดิสก์ 1 ลูก → (4 − 1) × 2 TB = **6 TB**

- **ตัวลวง b (4 TB):** คิดแบบ RAID 1/10
- **ตัวลวง d (8 TB):** คิดแบบ RAID 0

Part 2: ตอบสั้น (เฉลยละเอียด + เกณฑ์ 2 คะแนน)

S1. Full adder

- **Sum = A ⊕ B ⊕ Cin** (1 คะแนน)
- **Cout = AB + Cin(A ⊕ B)** หรือเขียนแบบ majority function = AB + ACin + BCin (1 คะแนน)

ที่มา: Sum เป็น 1 เมื่อจำนวน 1 ในอินพุตเป็นเลขคี่ (XOR) ส่วน carry เกิดเมื่อ A, B เป็น 1 ทั้งคู่ (generate) หรือเมื่อมีตัวใดตัวหนึ่งเป็น 1 แล้ว Cin = 1 (propagate)

S2. Moore "101" overlapping → 4 states, 2 flip-flops

State	ความหมาย	input 0 →	input 1 →	output
S0	ยังไม่เห็นอะไร	S0	S1	0
S1	เห็น "1"	S2	S1	0
S2	เห็น "10"	S0	S3	0
S3	เห็น "101"	S2	S1	**1**

- จาก S3 ได้ 0 ไป S2 (เพราะ "1" ตัวท้ายใช้ซ้ำได้ แล้วตามด้วย 0 = "10") และได้ 1 ไป S1
- **FF ขั้นต่ำ = $\lceil \log_2 4 \rceil = 2$** (ถ้าเป็น Mealy จะใช้แค่ 3 states)
- **เกณฑ์:** จำนวน state 1 คะแนน และจำนวน FF 1 คะแนน

S3. CPI = 2.3, CPU time = 2.3 s

1. CPI = 0.4(1) + 0.3(4) + 0.1(3) + 0.2(2) = 0.4 + 1.2 + 0.3 + 0.4 = **2.3** (1 คะแนน)
2. CPU time = $2 \times 10^9 \times 2.3 / (2 \times 10^9)$ = **2.3 s** (1 คะแนน)

ข้อสังเกต: load มี 30% ของคำสั่งแต่กินเวลา 1.2/2.3 = 52% ถ้าจะปรับปรุง ควรเริ่มที่ load (make the common case fast)

S4. n = 9 เท่า

1. $1 / (0.1 + 0.9/n) = 5$
2. $\rightarrow 0.1 + 0.9/n = 0.2$
3. $\rightarrow 0.9/n = 0.1$
4. $\rightarrow$ **n = 9**

ข้อสังเกต: speedup สูงสุดที่เป็นไปได้คือ 1/0.1 = 10 เท่านั้น ต้องเร่งถึง 9 เท่าเพื่อได้แค่ 5 (diminishing returns)

S5. d เริ่มที่ 16 และไบต์สุดท้ายคือ 24

ตัวแปร	ขนาด	ต้องหารลงตัวด้วย	วางที่
a (char)	1	1	0
b (int)	4	4	4–7 (pad 1–3)
c (short)	2	2	8–9
d (double)	8	8	**16–23** (pad 10–15)
e (char)	1	1	**24**

รวมใช้ 25 bytes (ถ้าเป็น struct ใน C จะมี padding ท้ายให้ครบพหุคูณของ 8 รวมเป็น 32 bytes)

S6. CPI = 4.0, เร็วเท่ากัน

1. CPI = 0.2(5) + 0.1(4) + 0.5(4) + 0.2(3) = 1.0 + 0.4 + 2.0 + 0.6 = **4.0** (1 คะแนน)
2. เวลาต่อคำสั่ง: multi = 4.0 × 50 = **200 ns** และ single = 200 ns → **เร็วเท่ากัน** speedup = 1.0 (1 คะแนน)

บทเรียน: multi-cycle ไม่ได้เร็วกว่าเสมอ ต้องคิด CPI ถ่วงด้วย instruction mix ทุกครั้ง

S7. Dependencies + stall 1 cycle

คู่	Register	ชนิด	ปัญหาจริงใน 5-stage in-order ไหม
1 → 2	r1	**RAW (load-use)**	**ใช่ ต้อง stall 1 cycle**
2 → 3	r3	**RAW**	forwarding แก้ได้
2 → 4	r1	**WAR**	ไม่เป็นปัญหา (ADD#2 อ่านที่ ID ก่อนที่ #4 จะเขียนที่ WB เสมอ)
1 → 4	r1	**WAW**	ไม่เป็นปัญหา (WB เกิดตามลำดับ)

- เวลารวม = (5 + 4 – 1) + 1 = **9 cycles**
- **เกณฑ์:** ระบุ dependency ครบ 1 คะแนน และตอบ stall = 1 ได้ 1 คะแนน

S8. RAID 5 vs RAID 4 + อย่างน้อย 3 ลูก

- **RAID 4** เก็บ parity บน**ดิสก์เฉพาะลูกเดียว** การเขียนทุกครั้งต้องอัปเดตดิสก์นั้น จึงเป็น**คอขวด**
- **RAID 5 กระจาย parity ไปทุกดิสก์** การเขียนที่ไปคนละ stripe จึงอัปเดต parity คนละดิสก์พร้อมกันได้ (1 คะแนน)
- ต้องมีดิสก์อย่างน้อย **3 ลูก** (ข้อมูล 2 + parity 1) (1 คะแนน)

S9. tag 18, index 8, offset 6

1. offset = $\log_2 64$ = **6 bits**
2. #sets = 64 KB / (64 B × 4 ways) = 65536 / 256 = **256 sets** → index = $\log_2 256$ = **8 bits**
3. tag = 32 – 8 – 6 = **18 bits**

- **กับดัก:** ถ้าลืมหารด้วย 4 ways จะได้ 1024 lines → index 10 และ tag 16 (ผิด)
- **เกณฑ์:** ถูก 3 ค่าได้ 2 คะแนน ถูก 2 ค่าได้ 1 คะแนน

S10. AMAT = 3.48 cycles

1. Miss penalty ของ L1 = AMAT ของ L2 = 12 + 0.25 × 200 = 12 + 50 = **62 cycles** (1 คะแนน)
2. AMAT = 1 + 0.04 × 62 = 1 + 2.48 = **3.48 cycles** (1 คะแนน)

เทียบกับไม่มี L2: 1 + 0.04 × 200 = 9 cycles → L2 ลด AMAT ได้มากกว่า 2.5 เท่า

แปลผลคะแนนชุดที่ 1 (เต็ม 60)

คะแนน	ระดับ	ทำต่อ
51–60	พร้อมมาก	อ่าน "กับดักรวม" ใน Exam Notes ก่อนสอบ

คะแนน	ระดับ	ทำต่อ
42–50	พร้อม	ทบทวนข้อที่ผิด แล้วทำชุดที่ 2
30–41	ปานกลาง	อ่านบทที่ผิดบ่อยใน Exam Notes ใหม่ แล้วทำชุดนี้ซ้ำ
< 30	ต้องเร่ง	เริ่มจาก Formula Sheet แล้วไล่ตัวอย่างในแต่ละบท

เฉลยละเอียด ชุดที่ 2

Part A: เติมคำ

#	คำตอบ	อธิบาย
1	**software, hardware**	ISA คือสัญญาระหว่าง SW กับ HW
2	**IBM 360** (System/360)	ก่อนหน้านั้นโปรแกรมไม่ portable
3	**Organization** (microarchitecture)	ต่างกันที่จำนวน core, cache, Turbo Boost, Hyper-Threading
4	**Synthetic**	เขียนขึ้นให้เลียนแบบโปรแกรมจริง
5	**Geometric**	ratio ไม่มีหน่วย และ GM ให้ผล consistent กับทุก baseline
6	**0x12**	little endian: 0x200 = 34, **0x201 = 12**, 0x202 = FE, 0x203 = CA
7	**A mod s**	
8	**Auto-increment**, `D[i++]`	อ่าน M[R2] แล้วเพิ่ม R2
9	**16**, **26**	I: 6 + 5 + 5 + 16 และ J: 6 + 26
10	**Zero pad**, **Sign extend**	ORUI ให้ imm ไปอยู่ 16 บิตบน (ส่วน ORI ใช้ zero extend)
11	**LW**	10 + 40 + 40 + 20 + 10 + 60 + 40 + 10 = 230 ns
12	**5, 4, 3**	
13	**FSM (Finite State Machine)**, **Microprogram**	
14	**WAR**	i เขียนสิ่งที่ j อ่าน
15	**SIMD**, **MIMD**	
16	**Hyper-Threading**	
17	**DRAM**, **SRAM**	
18	**Conflict**	แก้ด้วย associativity
19	**Page fault**, write-**back**	write-through ไปดิสก์แพงเกินไป
20	**6**, **3**	

Part B: ปรนัย

B1. **c)** จำนวน cache level เป็นเรื่องของ **organization** ซึ่ง software มองไม่เห็นผ่าน ISA ส่วน a, b, d ล้วนเป็นสิ่งที่ ISA กำหนด (format, addressing mode, interrupt handling)

B2. **b)**

- $T_A = 8 \times 10^8 / (400 \times 10^6) =$ **2 s**
- $T_B = 5 \times 10^8 / (300 \times 10^6) =$ **1.67 s**
- → B เร็วกว่า 2 / 1.67 = **1.2 เท่า**

ไม่ต้องรู้ clock rate เพราะ MIPS รวมไว้แล้ว (ตัวลวง d) และนี่คือเหตุผลที่ **MIPS ใช้เทียบข้าม ISA ไม่ได้**

B3. **d)** Load-store มี fixed-length format และ CPI ใกล้เคียงกันทุกคำสั่ง จึงจัด stage ได้สม่ำเสมอ ส่วนแบบอื่นมี bottleneck (ACC, stack) หรือ CPI แปรผัน (memory-memory)

B4. **b)** Stack มีข้อดีคือ code density ดีและ compiler ง่าย (ตัวลวง a, d กลับกัน) แต่มีข้อเสียคือข้อมูลที่ต้องการไม่ได้อยู่ top of stack เสมอ และ stack เป็น bottleneck

B5. **b)** การเทียบมาก/น้อยดูจาก byte สูง (มี sign bit) ก่อน ซึ่ง big endian เก็บ byte สูงไว้ที่ address ต่ำ จึงอ่านได้ก่อนและตอบได้ทันที ส่วน little endian เหมาะกับ**การบวก** (ส่ง carry จาก byte ต่ำขึ้นไป)

B6. **b) 0x8014**

0x8000 + 5 × 4 = 0x8000 + 20 = 0x8000 + **0x14**

- **ตัวลวง a:** ลืมคูณขนาด
- **ตัวลวง c (0x8050):** เป็นคำตอบของ handler table ขนาด 16 B

B7. **d)** XOR R1,R1,R1 ไม่ต้องมี immediate ทำให้ encode สั้น และ HW ส่วนใหญ่รู้จัก idiom นี้ (สไลด์: "For most architecture, the recommended method is XOR")

B8. **b)** **Code motion** คือย้ายโค้ดที่ไม่เปลี่ยนค่าใน loop (loop-invariant) ออกนอก loop เป็น global optimization ส่วน strength reduction คือใช้ operation ที่ถูกกว่า (เช่น shift แทนคูณ)

B9. **b) SW** SW เป็นคำสั่งเดียวที่เขียน memory (mem_wr = 1) และไม่เขียน register (reg_wr = 0)

B10. c) **0xBEEF0000** ORUI ใช้ **zero_pad** ซึ่งให้ imm ไปอยู่ 16 บิตบน แล้ว OR กับ r0 (= 0) ได้ 0xBEEF0000

- **ตัวลวง a:** zero extend (ORI)
- **ตัวลวง b:** sign extend

B11. b) **0x0038**

PC + 4 = 0x44 แล้วบวก sign_ext(−3) × 4 = −12 = −0xC → 0x44 − 0xC = **0x38**

- **ตัวลวง a:** ลืมบวก 4
- **ตัวลวง c:** ลืมคูณ imm ด้วย 4

B12. b) **≤ 4** เวลาต่อคำสั่งต้องไม่มากกว่าเดิม: CPI × 60 ≤ 240 → **CPI ≤ 4** (แบบเดียวกับตัวอย่างในสไลด์ 210/45 ≈ 4.67)

B13. b)

- ตัวลวง a ผิด (pipeline **ไม่ลด latency**)
- c ผิด (fill/drain และ hazard ทำให้ไม่ถึง k พอดี)
- d ผิด (stage ที่ไม่สมดุลทำให้ speedup ลดลง)

B14. a) **Structural** ถ้ามี memory ตัวเดียว คำสั่งหนึ่ง fetch (IF) พร้อมกับอีกคำสั่งหนึ่งอ่านหรือเขียนข้อมูล (MEM) ได้ไม่ได้ เป็นการแย่ง resource → แก้ด้วยการเพิ่ม resource (แยก I-cache กับ D-cache)

B15. b)

Load-use: LW ได้ข้อมูลตอนจบ MEM แต่ ADD ต้องใช้ตอนต้น EX ใน cycle เดียวกัน → ต้อง stall 1 cycle ("ต้องมีไทม์แมชชีน")

- a และ d: ผล ALU พร้อมตอนจบ EX จึง forward ได้ทัน
- c: ไม่มี dependency

B16. b) delay slot คือนิยามใหม่ว่าคำสั่งหลัง branch ทำงานเสมอ ซึ่ง**ต้องอาศัย compiler** หาคำสั่งมาใส่ (ตัวลวง d ผิด) และไม่ใช่ branch prediction (ตัวลวง c)

B17. b) **504**

T = (k + (n − x)/x) = 5 + 998/2 = 5 + 499 = **504 T_c**

- **ตัวลวง a (1004):** pipeline ธรรมดา (5 + 999)
- speedup เทียบ pipeline ≈ 1004/504 ≈ 2 = x

B18. c)

- VLIW ใช้ **static scheduling** (ตัวลวง a ผิด)
- VLIW **ลด IC** (n → n/x) โดย CPI = 1 (ตัวลวง b กลับกัน)
- Intel ประกาศ EOL ของ Itanium ในปี 2019 (ตัวลวง d ผิด)

B19. b)

- Associativity ลด **conflict miss** แต่ต้องมี comparator n ตัวและ MUX ใหญ่ขึ้น จึง**อาจเพิ่ม hit time**
- compulsory miss ไม่ได้ลดลง (ตัวลวง a)
- capacity miss ไม่เป็นศูนย์ (ตัวลวง c)
- ยังต้องมี tag (ตัวลวง d)

B20. a) Block, File EBS = block storage (เหมือนดิสก์เปล่าให้ VM) ส่วน NAS/NFS = file storage

Part C: คำนวณและอธิบาย

C1 (8 คะแนน)

(a)

- CPI = 0.5(1) + 0.2(5) + 0.1(4) + 0.2(3) = 0.5 + 1.0 + 0.4 + 0.6 = **2.5** (1.5)
- CPU time = $10^9 \times 2.5 / (2.5 \times 10^9)$ = **1.0 s** (1.5)

(b)

- $CPI_{ใหม่}$ = 0.5 + 0.2(3) + 0.4 + 0.6 = **2.1**
- Speedup = 2.5 / 2.1 = **1.19** (2) (IC และ clock เท่าเดิมจึงตัดทิ้งได้)

(c)

- Amdahl ต้องใช้ **P = สัดส่วนของเวลา** ที่ใช้กับ load (1)
- เวลาของ load = 0.2 × 5 = 1.0 จาก 2.5 → **P = 0.4** (1)
- n = 5/3 → 1 / (0.6 + 0.4 × 3/5) = 1 / (0.6 + 0.24) = 1/0.84 = **1.19** ✓ (1)
- ถ้าใช้ P = 0.2 จะได้ 1/(0.8 + 0.12) = 1.087 ซึ่ง**ผิด** เพราะ load แต่ละคำสั่งใช้เวลามากกว่าคำสั่งเฉลี่ย สัดส่วนของ**คำสั่ง**จึงไม่เท่ากับสัดส่วนของ**เวลา**

C2 (5 คะแนน)

- **(a)** 1 / (0.3 + 0.7/8) = 1/0.3875 = **2.58** (2)
- **(b)** 1/0.3 = **3.33** (1)
- **(c)** (2)
  - (i) 64 cores: 1 / (0.3 + 0.7/64) = 1/0.3109 = **3.22**
  - (ii) serial เร็วขึ้น 2 เท่า: 1 / (0.15 + 0.0875) = 1/0.2375 = **4.21**
  - → **(ii) ดีกว่า** เพราะส่วน serial เป็นตัวจำกัด speedup (Amdahl) การเพิ่ม core ให้ผลลดลงเรื่อย ๆ (diminishing returns)

C3 (9 คะแนน)

(a) single cycle มี CPI = 1: 7 × 230 = **1610 ns** (2)

(b) (3)

คำสั่ง	LW	LW	ADD	ORI	SW	BEQ	JMP	รวม
Cycles	5	5	4	4	4	3	3	**28**

28 × 50 = **1400 ns** (CPI เฉลี่ย = 4.0 → เร็วกว่า single เพียง 1.15 เท่า)

(c) (2)

- $(k + n - 1) \times T_c = (5 + 7 - 1) \times 50 =$ **550 ns**
- speedup = 1610/550 = **2.93**
- ไม่ถึง 5 เพราะ n น้อย (เวลา fill มีผลมาก) และ 5 stages × 50 = 250 ns > 230 ns

(d) ตอบอย่างน้อย 2 ข้อ (2)

- **RAW (load-use)** บน r2: LW#2 → ADD (r1 จาก LW#1 ห่าง 2 คำสั่ง forward ได้)
- **RAW** บน r3: ADD → ORI
- **RAW** บน r4: ORI → SW
- **Control hazard:** BEQ และ JMP

C4 (10 คะแนน)

(a) (3)

- offset = $\log_2 32$ = **5 bits**
- #lines = 16 KB / 32 B = 512 → index = **9 bits**
- tag = 32 – 9 – 5 = **18 bits**

(b) 0x00012345 = 74,565 (3)

- **offset** = 74,565 mod 32 = **5**
- **index** = (74,565 mod 16,384) div 32 = 9,029 div 32 = **282**
- **tag** = 74,565 div 16,384 = **4**
- ตรวจด้วย binary: 0x12345 = `0001 0010 0011 0100 0101` → 5 บิตล่าง = 00101 = 5, 9 บิตถัดไป = 1 0001 1010 = 0x11A = 282, ที่เหลือ = 0100 = 4 ✓

(c) (2)

- 0x16345 = 0x12345 + 0x4000 (+16,384 = **ขนาด cache พอดี**)
- index = 282 และ offset = 5 **เท่าเดิม** แต่ tag = 5 ≠ 4
- → อยู่ line เดียวกันแต่ tag ต่างกัน = **conflict miss** ถ้าใช้สลับกันจะไล่กันออกไปมา (ping-pong)

(d) (2)

- #sets = 512/2 = 256 → **offset 5, index 8, tag 19**
- ใช้ **2 comparators** (หนึ่งตัวต่อ way)
- ทั้งสอง address อยู่ set 282 mod 256 = 26 ได้พร้อมกัน → conflict หายไป

C5 (6 คะแนน)

(a) (3)

- ทุกคำสั่งต้อง fetch 1 ครั้ง และ 30% มี data access เพิ่ม
- CPI = 1 + 1 × 0.02 × 100 + 0.3 × 0.05 × 100 = 1 + 2 + 1.5 = **4.5**
- → ช้ากว่า perfect cache **4.5 เท่า** (memory stall กินเวลา 3.5/4.5 ≈ 78%)

(b) (3)

- MissPenalty_{L1} = 10 + 0.2 × 100 = 30 cycles
- AMAT = 1 + 0.05 × 30 = **2.5 cycles**
- ถ้าไม่มี L2 จะได้ 1 + 0.05 × 100 = 6 cycles

C6 (10 คะแนน)

(a) Dependencies (3)

คู่	Register	ชนิด
1 → 2	r1	RAW
2 → 4	r3	RAW
3 → 4	r6	RAW

ไม่มี WAR/WAW (ไม่มีคำสั่งใดเขียน register ซ้ำ หรือเขียนทับสิ่งที่คำสั่งก่อนหน้าอ่าน) ถ้าตอบว่ามีจะเสียคะแนน

(b) (3)

- 2 → 4 (r3) เป็นผล ALU จึง forward ได้ไม่ต้อง stall
- 1 → 2 และ 3 → 4 เป็น **load-use** → stall จุดละ 1 cycle
- ideal = 5 + 4 = 9 cycles → รวม **11 cycles**

(c) (3)

```
1: LW   $r1, 0($r2)
3: LW   $r6, 4($r2)
2: ADD  $r3, $r1, $r4    ; ห่างจาก LW r1 สองคำสั่ง ใช้ forwarding ได้
5: ORI  $r8, $r9, #5     ; คำสั่งอิสระ คั่นระหว่าง LW r6 กับ SUB
4: SUB  $r7, $r6, $r3    ; r3 forward จาก ADD และ r6 พร้อมแล้ว
```

- → **9 cycles ไม่มี stall** (speedup 11/9 = 1.22)
- ตรวจแล้วว่าไม่สร้าง WAR/WAW ใหม่ (ORI เขียน r8 ซึ่งไม่มีใครใช้)
- ลำดับอื่นที่ถูกก็ได้คะแนน เช่น LW, LW, ORI, ADD, SUB

(d) (1) **Compiler scheduling / delayed load**: ย้ายคำสั่งอิสระมาคั่น ถ้าไม่มีให้แทรก **NOP** ให้ครบจำนวนที่ต้องรอ (สไลด์: `LW; NOP; NOP; NOP; AND`)

C7 (6 คะแนน)

(a) (2)

- offset = $\log_2 4096$ = **12 bits**
- VPN = 32 − 12 = **20 bits**
- PA = $\log_2$(1 GB) = 30 bits → PPN = 30 − 12 = **18 bits**

(b) $2^{20} \times 4$ B = **4 MB ต่อ process** (1)

(c) (2)

- overlap ได้เมื่อ **index + offset ≤ page offset (12 bits)** หรือก็คือ cache size / ways ≤ page size
- 32 KB / 4 KB = **8-way** (แต่ละ way 4 KB ดังนั้นบิตที่ใช้ index ทั้งหมดอยู่ใน page offset ซึ่งไม่เปลี่ยนตอนแปล)
- ทางเลือกอื่น: page 32 KB หรือให้ OS รับประกันว่าบิต 14–12 ของ VA = PA (page coloring)

(d) (1)

- **TLB: flush** (หรือใช้ ASID/PID tag) เพราะ mapping เป็นของ process เก่า
- **Page table: ไม่ต้องลบ** แค่เปลี่ยน page table register ให้ชี้ไปที่ table ของ process ใหม่

C8 (เลือก 2 ข้อ ข้อละ 3 คะแนน)

(a) RAID 5 vs RAID 4

- RAID 4 เก็บ parity บน**ดิสก์เฉพาะลูกเดียว** การเขียนทุกครั้งต้องอัปเดตดิสก์นั้น จึงเป็น **bottleneck**
- RAID 5 **กระจาย parity ไปทุกดิสก์** การเขียนหลาย stripe จึงทำขนานกันได้ (read ก็ใช้ได้ครบทุกดิสก์)
- ทั้งคู่ทนดิสก์เสียได้ 1 ลูก
- **RAID 0** เหมาะกับงานที่ต้องการ**ความเร็วสูงสุด**และข้อมูลไม่สำคัญหรือมี backup อยู่แล้ว (scratch, temp ของงานตัดต่อวิดีโอ) เพราะเสีย 1 ลูกข้อมูลหายหมด

(b) Write-back vs Write-through

- **Write-back:** เขียนเฉพาะ cache (ตั้ง dirty bit) แล้วค่อยเขียนกลับตอน replace → **เร็ว ดีกับการเขียนซ้ำ** แต่**ใน multiprocessor ข้อมูลจะ incoherent**
- **Write-through:** เขียนทั้งสองที่ทันที → **ช้า** แต่ memory เป็นปัจจุบันเสมอ **เหมาะกับ multiprocessor**
- **Write buffer:** FIFO ที่ทำให้ CPU ไม่ต้องรอ memory ตอน write-through
- **Saturate** เมื่ออัตราการเขียนสูงกว่าที่ memory รับไหว ซึ่งจะเต็มไม่ว่า buffer ใหญ่แค่ไหน → แก้ด้วย **L2 cache**

(c) Row-major

- **แบบ X เร็วกว่า** เพราะ C เก็บ array แบบ **row-major** (`data[r][0], data[r][1], …` อยู่ติดกัน) X จึงเดินเรียง address ทีละ 4 bytes ทำให้ได้ **spatial locality** (1 block มีหลาย element จึง hit ต่อเนื่อง)
- แบบ Y กระโดดทีละ 4000 bytes เกือบทุกครั้งตกคนละ block ทำให้ miss บ่อย และ block อาจถูกไล่ออกก่อนจะได้ใช้ element ถัดไป

(d) Dynamic vs Static

- **Dynamic:** HW ซับซ้อนและใช้ power มาก แต่ **binary เดิมรันได้และเร็วขึ้นบน HW ใหม่** (ไม่ต้อง recompile) และรู้ dependency จริงตอนรัน
- **Static (VLIW):** HW ง่ายและประหยัด power, compiler เห็น scope กว้าง แต่ **binary ผูกกับรุ่น HW** (Itanium → Itanium II ไม่ compatible) และ compiler รู้ dependency บางอย่างไม่ได้ (pointer alias, cache miss)
- **ในทางปฏิบัติ dynamic ชนะ** เพราะ compatibility และ "proven to be too difficult to have good compiler" (Itanium ถูก EOL ในปี 2019)

แปลผลคะแนนชุดที่ 2 (เต็ม 100)

คะแนน	ระดับ	ทำต่อ
85–100	พร้อมมาก	ทบทวนกับดักรวม
70–84	พร้อม	ทำ Exercise ท้ายบทที่ผิดซ้ำ
50–69	ปานกลาง	อ่าน Ch2, Ch6, Ch8 ใน Exam Notes ใหม่
< 50	ต้องเร่ง	เริ่มจาก Formula Sheet แล้วไล่ตัวอย่างทุกบท